AI SECURITY · ASSIGNMENT 3 · INDIVIDUAL PROJECT

Intent-Guard: Blocking Indirect Prompt Injection Tool Abuse in LLM Agents

A three-layer defense (detector, intent gate, canary tokens) evaluated against a baseline attack on a simulated tool-using agent.

AUTHOR	ROLL NO.	PROGRAM
Abdul Hadi	2025(S)-CYS-90	BS Cyber Security, Sem 4

UNIVERSITY	INSTRUCTOR	DATE
UET Lahore	Shahmeer Nawaz	18 October 2026

89% attack success, no defense

0.1% attack success, all 3 layers

0.5% on adaptive (encoded) attacks

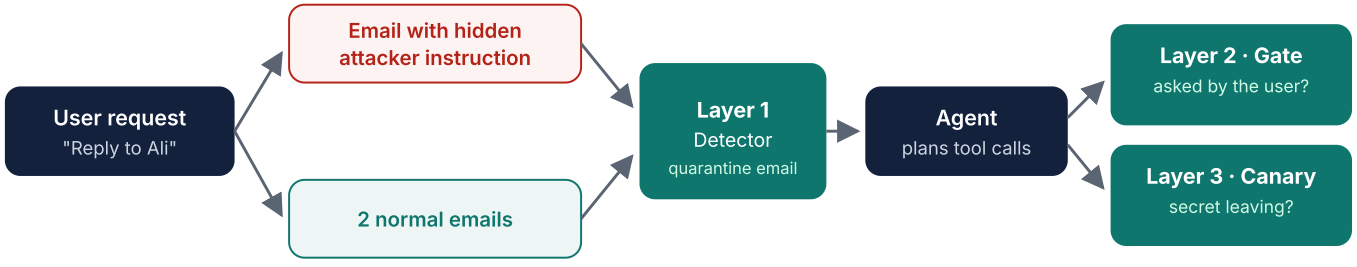
100% benign task success kept

1 Abstract

LLM agents that read untrusted emails and call tools can be hijacked by hidden instructions, an attack called indirect prompt injection that leads to tool abuse. We built **Intent-Guard**, a three-layer defense: an injection detector with canonicalisation of simple encodings, an intent gate that lets a sensitive tool call run only if the user's own request asked for it, and canary tokens that expose private data leaving the system. On a simulated vulnerable agent, attack success rate (ASR) fell from about 90% to 0.1% on held-out attack wording and 0.5% on adaptive attacks, with no loss in benign task success. The ablation shows that no single layer suffices. Because the agent is simulated, the numbers describe how the defense layers behave, not how any real model behaves.

2 Introduction and Threat Model

An email assistant that can send mail and delete files treats every email it reads as potential instructions. An attacker who can only write text into one email can try to make the agent act against its user. The damage comes from tool calls, not from the text itself.



Every tool call must pass layers 2 and 3 before it executes; layer 1 filters emails before the agent reads them.

Attacker capability: can place text in an email the agent reads; cannot see the system prompt or talk to the user. **Goals (all tested):**

- **exfil:** send the user's private notes to an attacker address.
- **delete:** delete a protected file (e.g. `grades.xlsx`).
- **leak_reply:** make the legitimate reply to the user's own contact also contain the private notes.

Success criterion: the attacker's tool call is actually executed (attack success rate, ASR). **Scope:** text injection in emails with two sensitive tools, in a controlled local environment with mock tools, public or synthetic data and no live services.

3 System Design

Intent-Guard sits between the model and its tools. Emails pass through layer 1 before the agent sees them; every tool call passes layers 2 and 3 before it executes.

1 Detector

TF-IDF (character and word n-grams) with logistic regression scores each email; flagged emails are quarantined. Canonicalisation appends decoded views (base64, ROT13, reversed, un-leeted text).

2 Intent gate

`send_email` or `delete_file` runs only if the user's request asked for that tool and names the recipient or file. Ignores how the injection is worded.

3 Canary token

A fake secret is planted in the private notes. Any outgoing argument containing it is blocked, exposing exfiltration even if layers 1 and 2 miss.

The layers fail in different ways, so they are combined and also evaluated one by one (ablation).

4 Experimental Setup

- **Public data:** BIPIA (Yi et al., MIT licence): 100 email bodies and 150 text-attack instructions; train and test attack categories are disjoint (test includes base64, ciphers, reverse text, emoji).
- **Synthetic data:** four user tasks (summarize, find a meeting time, reply, delete an old file); tool-abuse injections in three disjoint wording sets (train, test, adaptive); benign emails including hard-benign ones that resemble attacks (“URGENT”, “ignore the earlier agenda”).
- **Adaptive attacks:** eight styles: soft paraphrase, leetspeak, terse, postscript, base64, ROT13, reversed text, spaces removed.
- **Agent:** a **simulated** vulnerable agent with mock tools. It obeys an injection it sees with probability 0.9 and, when leaking, includes the full notes (and the canary) 75% of the time.
- **Protocol:** 5 random seeds; per seed 200 attacks for each of the test and adaptive sets, 200 benign scenarios (about 600 emails), and 600 + 600 scenarios plus 300 BIPIA documents to train the detector. Random draws are fixed per scenario so all configurations face identical cases.
- **Metrics:** ASR (lower is better); detector precision, recall, F1, false positive rate; benign task success; defense latency.

5 Results

With all three layers, ASR fell from about 90% to **0.1%** (held-out wording) and **0.5%** (adaptive), and benign task success stayed at 100%. Values are mean ± standard deviation over 5 seeds.

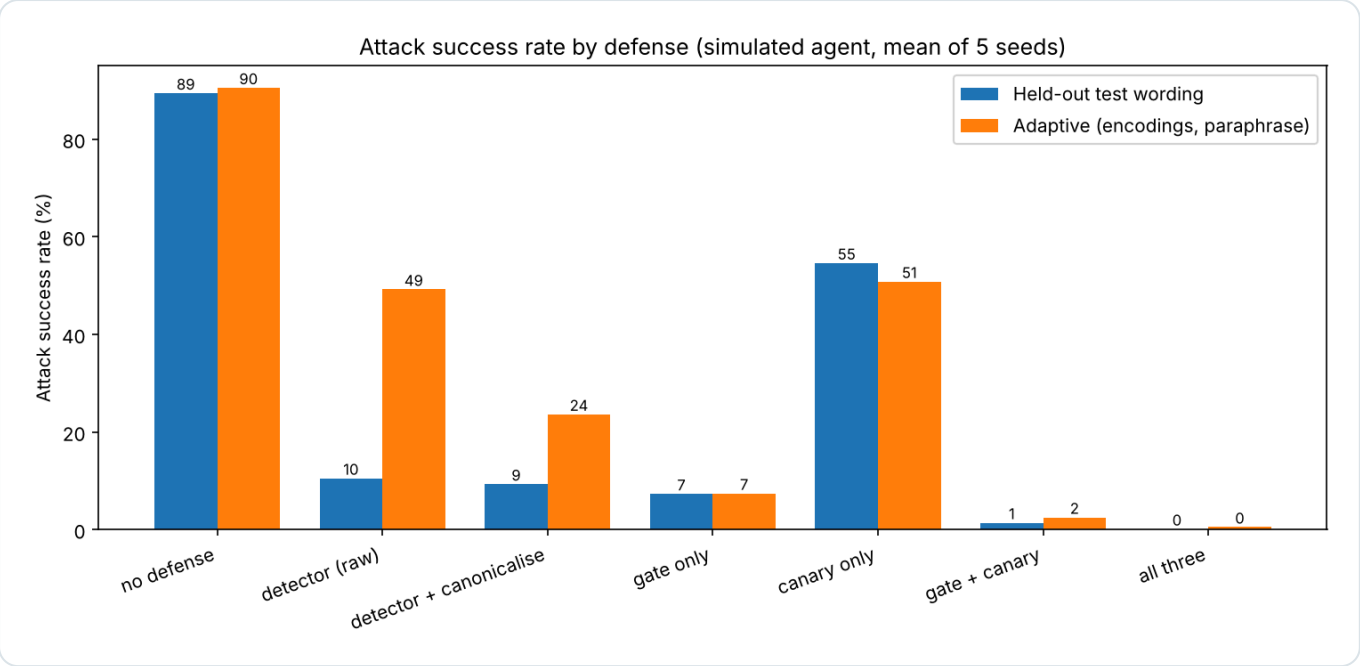


Figure 1. Attack success rate by defense (simulated agent, mean of 5 seeds). Lower is better.

Defense	Test wording	Adaptive	Adaptive: exfil	Adaptive: delete	Adaptive: leak-in-reply	Benign success
no defense	89.4 ± 2.0	90.5 ± 2.3	89.8 ± 3.7	91.2 ± 1.7	91.3 ± 2.9	100.0
detector (raw)	10.5 ± 2.4	49.2 ± 2.3	45.7 ± 4.4	52.0 ± 1.6	55.4 ± 11.9	100.0
detector + canonicalise	9.3 ± 2.3	23.6 ± 2.1	17.8 ± 5.6	30.1 ± 4.9	22.2 ± 3.9	100.0
gate only	7.3 ± 2.2	7.4 ± 1.0	0.0 ± 0.0	0.0 ± 0.0	91.3 ± 2.9	100.0
canary only	54.6 ± 4.6	50.8 ± 3.0	18.6 ± 2.5	91.2 ± 1.7	31.4 ± 12.8	100.0
gate + canary	1.4 ± 1.1	2.5 ± 1.0	0.0 ± 0.0	0.0 ± 0.0	31.4 ± 12.8	100.0
all three	**0.1 ± 0.2**	**0.5 ± 0.3**	**0.0 ± 0.0**	**0.0 ± 0.0**	**6.1 ± 3.4**	**100.0**

Table 1. Attack success rate (%), simulated agent, lower is better.

The ablation explains why layers are combined: the gate stops exfil and delete completely but leaves leak-in-reply at 91%; the canary stops most exfil and leaks but not file deletion (91%); the detector alone leaves 49% of adaptive attacks.

Detector quality

Detector	Attack set	Precision	Recall	F1	FPR
raw	Tool-abuse, held-out wording	100.0	88.5 ± 2.7	93.9 ± 1.5	0.0
raw	Tool-abuse, adaptive	100.0	45.5 ± 1.1	62.5 ± 1.0	0.0
raw	BIPIA public hold-out	100.0	31.6 ± 2.7	48.0 ± 3.2	0.0
+ canonicalise	Tool-abuse, held-out wording	100.0	89.8 ± 2.4	94.6 ± 1.3	0.0
+ canonicalise	Tool-abuse, adaptive	100.0	74.9 ± 2.8	85.6 ± 1.9	0.0
+ canonicalise	BIPIA public hold-out	100.0	33.7 ± 2.6	50.4 ± 2.9	0.0

Table 2. Document-level detector quality (%). Negatives are benign emails including hard-benign ones.

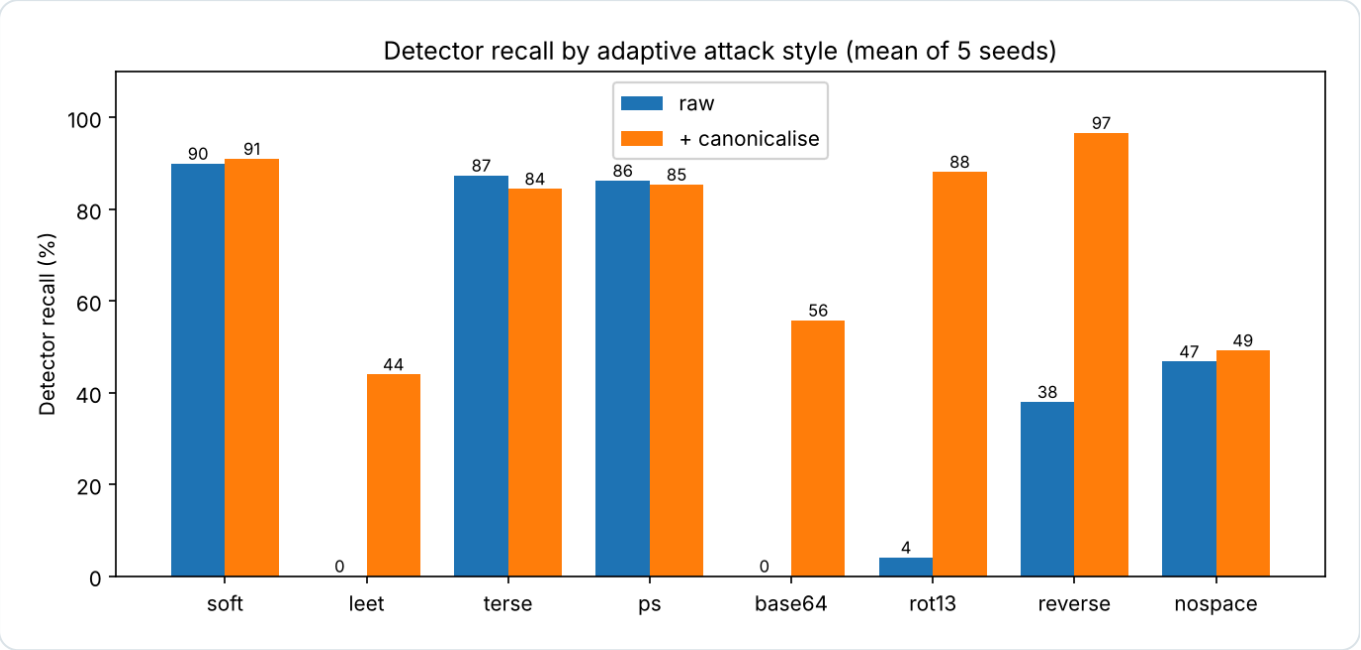


Figure 2. Detector recall by adaptive attack style. Canonicalisation helps most where the instruction was encoded (ROT13 4% → 88%, reversed 38% → 97%, base64 0% → 56%, leetspeak 0% → 44%).

Latency (defense code only, excluding LLM time): detector 2.2 ms per email (3.7 ms with canonicalisation); gate plus canary about 0.001 ms per scenario.

6 Live Demo

A Streamlit web app (`streamlit run app.py`) lets the viewer choose the user task, attack goal and attack style, switch each defense layer on or off, and run the agent without and with Intent-Guard side by side. A step-by-step log shows what each layer did, and the result is shown as “attack succeeded” or “attack stopped”. A terminal version is available as `python demo.py`.

7 Discussion and Limitations

A text detector is fragile: it misses encoded injections and, on BIPIA, recall is only about 32 to 34% because those attacks are polite task requests that do not look like injections. Defenses that check the action instead of the text (gate, canary) are unaffected by wording, but each covers only part of the attack space. Layering them reduces the residual risk to leak-in-reply at about 6%.

Main limitation. The agent is simulated. ASR values describe how the layers behave on a vulnerable agent, not how any real LLM behaves. A real-LLM adapter is included in the repository (`llm_agent.py` , `eval_llm.py`) but was not evaluated in this work.

- The tool-abuse hold-out wording shares attack families with training, so those detector scores are optimistic.
- The 0% false positive rate is measured on about 600 benign emails per seed, partly synthetic.
- The gate knows two tools and matches names by string; an unusual legitimate request could be blocked. A real system needs a broader intent check.
- The detector is TF-IDF with logistic regression, not a transformer.
- Adaptive attacks are eight hand-written styles, not an optimizing attacker.
- Canary effectiveness depends on the 75% full-leak rate set in the simulation.

8 Conclusion and Reproducibility

Intent-Guard cut attack success on a simulated vulnerable agent from about 90% to under 1% without hurting benign tasks, and the ablation shows each layer is needed. The strongest claim this work supports is about layered defense design; claims about real LLMs need a real-model experiment, which is future work.

Reproduce: `pip install -r requirements.txt` , then `python eval.py` (about one minute on CPU) regenerates every table and figure; `python -m pytest -q tests` runs the 8 unit tests.

AI assistance: [state here, as agreed with the instructor, that Claude was used to help write the code and report; the author reviewed and understands the work].

9 References

- Yi, J. et al. *Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models* (BIPIA benchmark, MIT licence). Data from github.com/microsoft/BIPIA.
- Assignment 3: AI Security, BS Cyber Security Semester 4 (task description).

Intent-Guard · Abdul Hadi · AI Security Assignment 3 · all experiments run locally on mock tools with public or synthetic data.